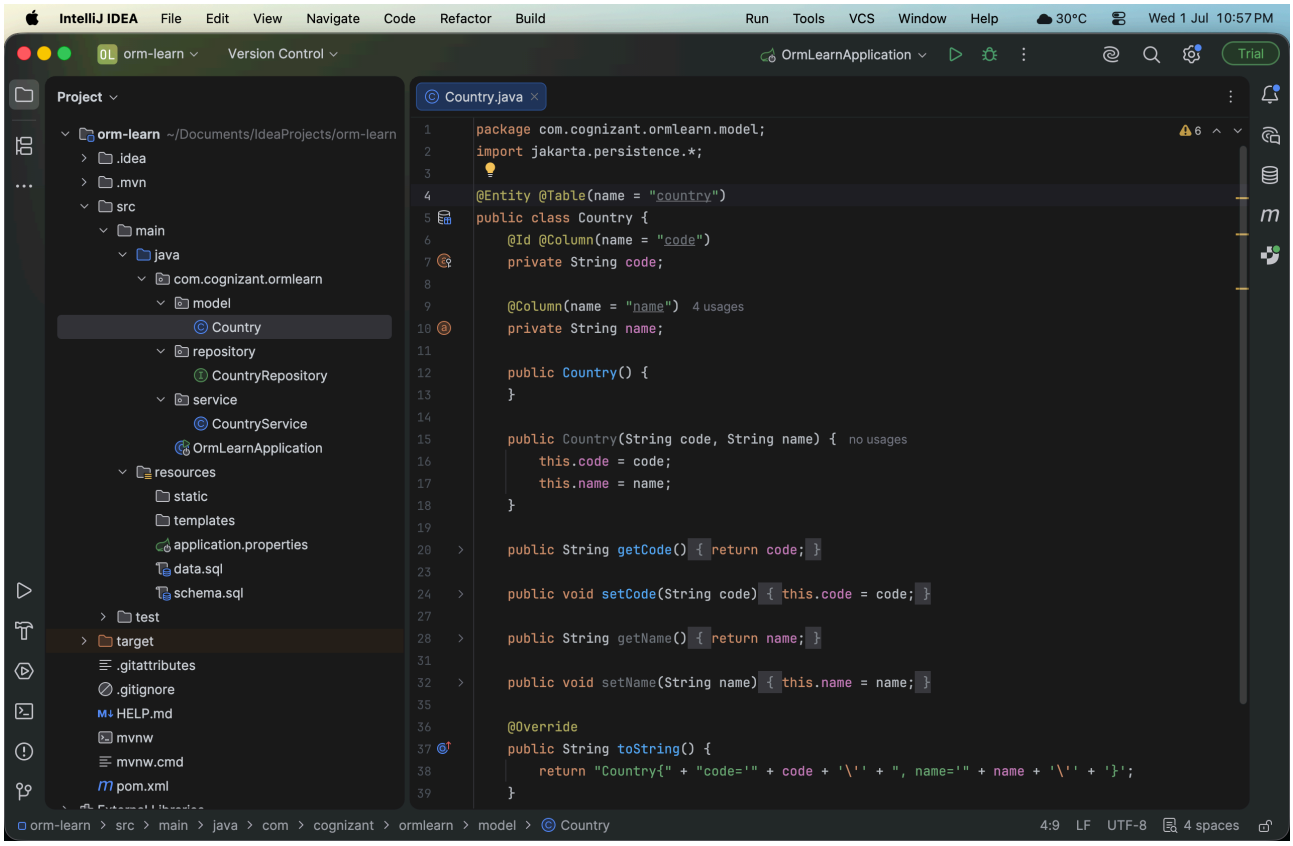
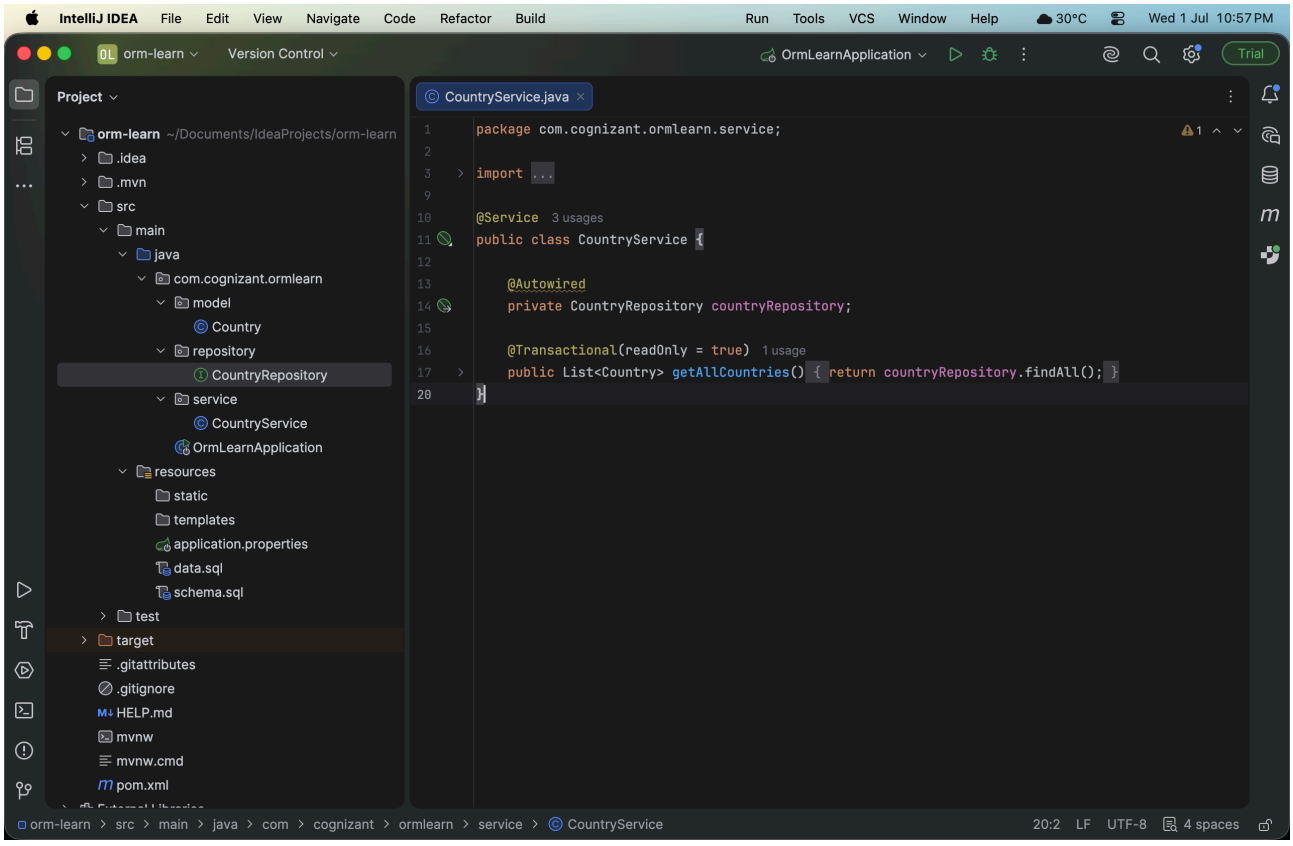
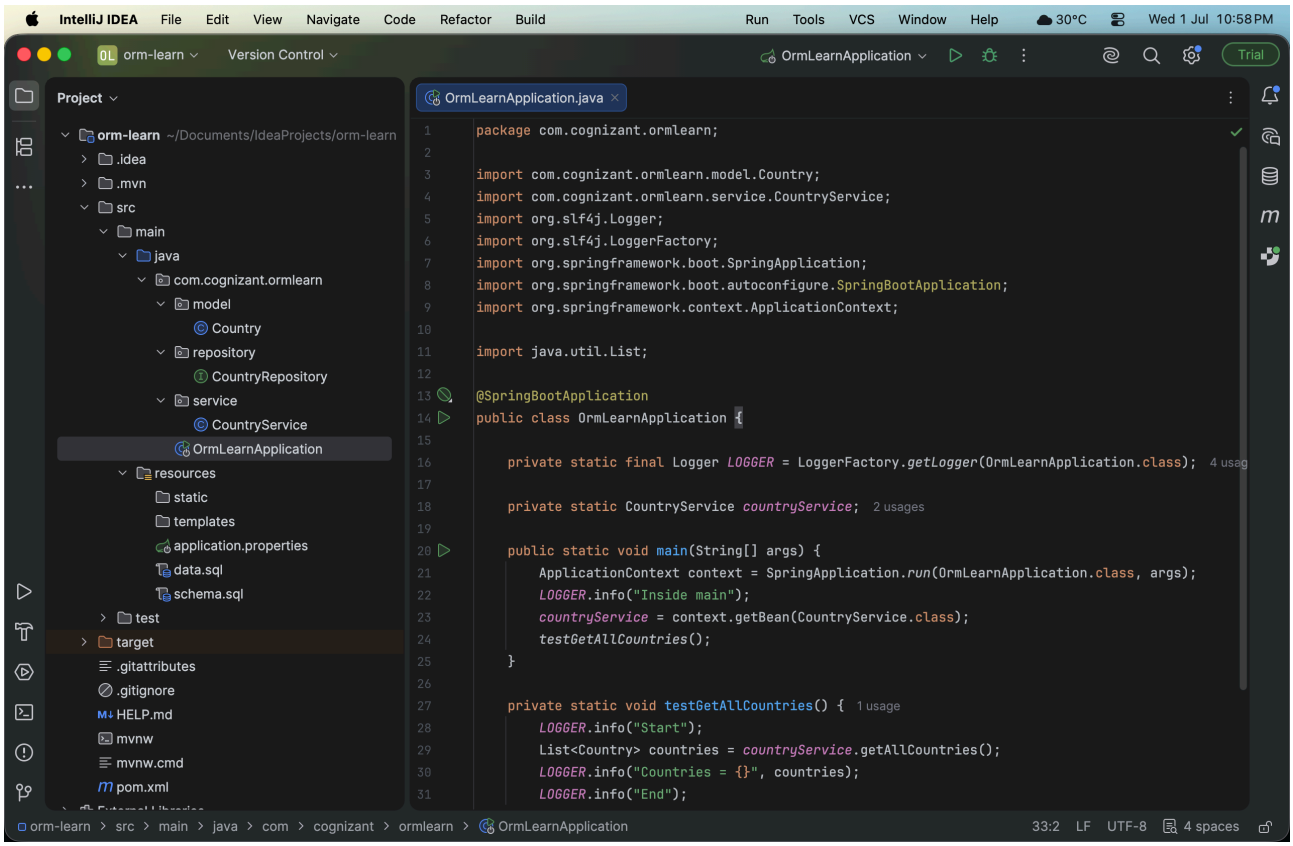
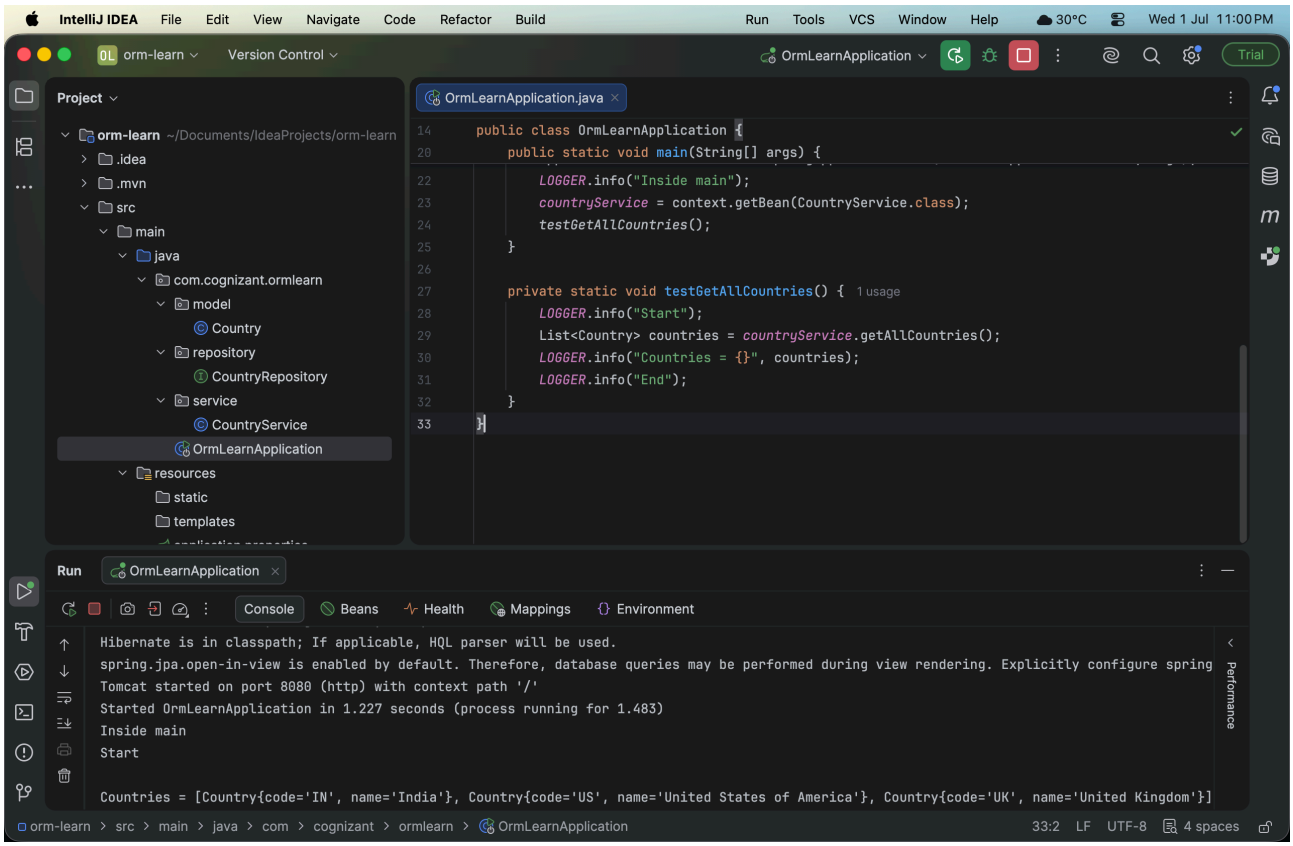
Spring Data JPA

Hands on 1: Spring Data JPA - Quick Example






IntelliJ IDEA File Edit View Navigate Code Refactor Build Run Tools VCS Window Help 30°C Wed 1 Jul 10:58 PM
orm-learn Version Control OrmLearnApplication Trial
Project
orm-learn ~/Documents/IdeaProjects/orm-learn
.idea
.mvn
src
main
java
com.cognizant.ormlearn
model
Country
repository
CountryRepository
service
CountryService
OrmLearnApplication
resources
static
templates
application.properties
data.sql
schema.sql
test
target
.gitattributes
.gitignore
HELP.md
mvnw
mvnw.cmd
pom.xml
OrmLearnApplication.java
package com.cognizant.ormlearn;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.service.CountryService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import java.util.List;
@SpringBootApplication
public class OrmLearnApplication {
private static final Logger LOGGER = LoggerFactory.getLogger(OrmLearnApplication.class); 4 usag
private static CountryService countryService; 2 usages
public static void main(String[] args) {
ApplicationContext context = SpringApplication.run(OrmLearnApplication.class, args);
LOGGER.info("Inside main");
countryService = context.getBean(CountryService.class);
testGetAllCountries();
}
private static void testGetAllCountries() { 1 usage
LOGGER.info("Start");
List<Country> countries = countryService.getAllCountries();
LOGGER.info("Countries = {}", countries);
LOGGER.info("End");
orm-learn > src > main > java > com > cognizant > ormlearn > OrmLearnApplication
33:2 LF UTF-8 4 spaces


IntelliJ IDEA File Edit View Navigate Code Refactor Build Run Tools VCS Window Help 30°C Wed 1 Jul 11:00 PM
orm-learn Version Control OrmLearnApplication Trial
Project
orm-learn ~/Documents/IdeaProjects/orm-learn
.idea
.mvn
src
main
java
com.cognizant.ormlearn
model
Country
repository
CountryRepository
service
CountryService
OrmLearnApplication
resources
static
templates
OrmLearnApplication.java
public class OrmLearnApplication {
public static void main(String[] args) {
LOGGER.info("Inside main");
countryService = context.getBean(CountryService.class);
testGetAllCountries();
}
private static void testGetAllCountries() { 1 usage
LOGGER.info("Start");
List<Country> countries = countryService.getAllCountries();
LOGGER.info("Countries = {}", countries);
LOGGER.info("End");
}
}
Run OrmLearnApplication
Console Beans Health Mappings Environment
Hibernate is in classpath; If applicable, HQL parser will be used.
spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering. Explicitly configure spring
Tomcat started on port 8080 (http) with context path '/'
Started OrmLearnApplication in 1.227 seconds (process running for 1.483)
Inside main
Start
Countries = [Country{code='IN', name='India'}, Country{code='US', name='United States of America'}, Country{code='UK', name='United Kingdom'}]
Performance
orm-learn > src > main > java > com > cognizant > ormlearn > OrmLearnApplication
33:2 LF UTF-8 4 spaces

Hands on 4: Difference between JPA, Hibernate and Spring Data JPA

1. JPA (Java Persistence API)

- JPA is a specification (standard).
- It defines how Java objects are mapped to database tables.
- It does not contain any implementation.
- It requires an implementation such as Hibernate.

Example:

```
@Entity
public class Country {
   @Id
   private String code;

   private String name;
}
```

2. Hibernate

- Hibernate is an ORM framework.
- It implements JPA.
- It converts Java objects into database records.

Example:

```
Session session = factory.openSession();
Transaction tx = session.beginTransaction();
session.save(employee);
tx.commit();
session.close();
```

3. Spring Data JPA

Spring Data JPA sits on top of Hibernate.
Instead of writing lots of code, you only create a repository.

Example:

Repository

```
@Repository
public interface EmployeeRepository extends JpaRepository<Employee,Integer> {}
```

Service

```
@Service
public class EmployeeService {
    @Autowired
    private EmployeeRepository employeeRepository;

    @Transactional
    public void addEmployee(Employee employee){
        employeeRepository.save(employee);
    }
}
```

Advantages of Spring Data JPA

- Less code
- Easy CRUD operations
- Automatic transaction management
- Easy integration with Spring Boot
- Supports custom queries
- Built-in pagination and sorting